

# TP Virtualization

Teabe Boris, [boris.teabe@inp-toulouse.fr](mailto:boris.teabe@inp-toulouse.fr)

The goal of this TP is to experiment with virtualization.

## Learning objectives

The goal of this lab is to experiment with virtualization using Xen. At the end of the lab, you should be able to:

- explain the role of a hypervisor and distinguish Dom0 from DomU in Xen;
- configure bridged networking for virtual machines;
- create, start, inspect and access a Xen virtual machine;
- explain why live migration requires the destination host to access the VM storage;
- perform a live migration between two Xen hosts;
- measure the impact of live migration on network and application availability.

## 1. Installation

A large physical server runs KVM. One KVM virtual machine has been created for each student group. During this lab, this KVM VM must be considered as your physical server. You will install Xen inside it and create another VM (DomU) managed by Xen. This is therefore a nested virtualization environment.

```
Physical server
├── KVM
│   ├── VM Group 01 ("server1")
│   │   ├── Xen
│   │   │   ├── Dom0
│   │   │   └── DomU: myvm
│   └── VM Group 02 ("server2")
│       ├── Xen
│       └── Dom0
```

Use the following command to access your server:

**ssh ubuntu@147.127.121.11 -p 130XX**

where XX is your group number (01–40). Your server uses an address in the 192.168.27.0/24 network. Record its IP address before continuing.

*ip addr*

*ip route*

**Q1. Identify the three virtualization layers in the experimental setup. Which layer**

corresponds to the infrastructure provided by the teaching staff, and which layer will you configure yourself?

## 2 Installation of Xen in your server

- install ssh in your server / the ssh-server can already be installed if you use the VM we provided.

```
sudo bash
```

```
apt-get update
```

```
sudo apt-get install openssh-server -y
```

- enable root login on your ssh server
  - edit `/etc/ssh/sshd_config` (using `vim` or `nano`)
  - configure

```
PermitRootLogin yes
```

- reload your configuration

```
service ssh reload
```

- provide a password for root

```
passwd root
```

- install bridge tools

```
apt-get install -y bridge-utils
```

- install Xen hypervisor

```
sudo apt-get install -y xen-hypervisor-amd64
```

- update grub to boot the hypervisor and reboot

```
cd /etc/default/grub.d/
```

- modify the file « `xen.cfg` » and edit line **83** to set :

```
XEN_OVERRIDE_GRUB_DEFAULT=1
```

- and then update the grub

```
update-grub
```

```
reboot
```

- Reconnect to your working environment by redoing the ssh as at the beginning of the class.
- verify the Xen is working properly

```
sudo bash
```

```
xl list
```

- you should see one VM: the **dom0**

At this stage, `xl list` should show Dom0 even though you have not created a guest VM yet.

**Q2. What is Dom0? Why does it appear in `xl list` before you create any VM?**

**Q3. Is Xen a type-1 or type-2 hypervisor? Explain where Xen is located relative to Linux in this setup.**

**Q4. Using `xl info`, record the number of CPUs and the amount of memory visible to Xen.**

## 3. Configure Xen networking

Before changing the network configuration, inspect the current state:

```
ip addr
```

*ip route*

*ip link*

Create a Linux bridge named xenbr0. Adapt the interface name and IP address to your server. On recent Netplan versions, use a default route instead of the deprecated gateway4 field.

- update your network configuration to add a bridge
  - nano **/etc/netplan/50-cloud-init.yaml (with vim or nano)**
  - set this content :

**network:**

**ethernets:**

**enp1s0:**

**dhcp4: no**

**bridges:**

**xenbr0:**

**interfaces: [enp1s0]**

**addresses: [<current IP address>/24]**

**gateway4: 192.168.27.1**

**nameservers:**

**addresses: [8.8.8.8]**

**version: 2**

Test your configuration

**netplan try**

If everything is ok, press « Enter » to accept the new network configuration

Verify that networking is available

**ping 8.8.8.8**

**ping google.fr**

**Q5. Why is the server IP address assigned to xenbr0 rather than directly to enp1s0?**

**Q6. Draw the expected network path of a packet sent from a Xen VM to the external network.**

## **4. Creation of a VM image**

Download an Ubuntu cloud image. The example below uses Ubuntu 20.04 because it is compatible with the configuration used in this lab.

*cd /home/ubuntu*

*wget http://cloud-images.ubuntu.com/releases/focal/release/ubuntu-20.04-server-cloudimg-amd64.img -P /home/ubuntu/ -O vm.qcow2*

We need to create the VM configuration file

**cat << EOF > /etc/xen/vm.cfg**

**bootloader = 'pygrub'**

**vcpus = 2**

**memory = 1024**

**root = '/dev/xvda1 ro'**

**disk = [**

```

        '/home/ubuntu/vm.qcow2,qcow2,hda,rw'
    ]
    name      = 'myvm'
    vif = [ 'bridge=xenbr0' ]
EOF

```

Important parameters:

- vcpus: number of virtual CPUs assigned to the guest;
- memory: RAM allocated to the guest, in MB;
- disk: virtual block device backing the guest;
- vif: virtual network interface;
- bridge: Linux bridge to which the guest interface is attached;
- name: Xen domain name.

Modify the default password of the downloaded VM image

```

modprobe nbd max_part=8
qemu-nbd --connect=/dev/nbd0 /home/ubuntu/vm.qcow2
mount /dev/nbd0p1 /mnt/
chroot /mnt/

```

Change the vm password with this command, and you should type the new password. **Don't forget the new password.**

```

    passwd
    exit

```

Type the new root password for the VM and then.

```

umount /mnt/
qemu-nbd --disconnect /dev/nbd0
rmmod nbd

```

## 5. Creation of a VM

Create the VM

```

xl create /etc/xen/vm.cfg

```

Verify that the VM was created

```

xl list

```

You should see your domU

To connect to the console of the VM use the command, username root **password you mus know**

```

xl console myvm

```

You need to configure the network in the VM (set the VM IP address add your IP address + 100 on the last number) as we did previously.

```

cat << EOF > /etc/netplan/50-cloud-init.yaml

```

```

network:

```

```

    ethernets:

```

```

        eth0:

```

```

            dhcp4: no

```

```

            addresses: [<current IP address + 100>/24]

```

**gateway4: 192.168.27.1**

**nameservers:**

**addresses: [8.8.8.8]**

**version: 2**

**EOF**

-Test your configuration

**netplan try**

- If everything is ok, press « Enter » to accept the new network configuration

- verify that networking is available

**ping 8.8.8.8**

**ping google.fr**

**- To disconnect from the console use this keyboard combinaison “Ctrl + J”**

**Q7. Compare the memory configured in vm.cfg with the memory visible from inside the VM.**

**Q8. From Dom0, use ip link and bridge link to identify the virtual interface corresponding to myvm.**

## **6. Experiment with migration**

Work with another group. The host initially running myvm is server1; the partner host is server2.

Both servers must have Xen configured and must be able to communicate over the network.

Before configuring shared storage, discuss what will happen if server2 cannot access the disk image of myvm.

**Q9. During a live migration, which parts of the VM state must move from server1 to server2? Which part can remain on shared storage?**

To migrate **myvm** to **server2**, the filesystem of **myvm** (in vms) should be accessible in **server2**

Install NFS and mount the directory which includes the VM image

- in server1

**sudo apt-get install nfs-server nfs-common**

- edit **/etc/exports**

- add

**/home/ubuntu/ \*(rw,sync,no\_root\_squash)**

- restart NFS

**systemctl restart nfs-kernel-server**

**systemctl status nfs-kernel-server**

- in server2

**sudo apt-get install nfs-common**

**sudo mount -t nfs <IP address of server1>:/home/ubuntu/ /home/ubuntu**

- verify that the VM image is accessible from server2

**ls /home/ubuntu/**

- you should see the same content as on server1

- in server1, trigger the migration

**sudo xl migrate myvm <IP address of server2>**

- verify that migration was performed
    - in server1 and server2
- sudo xl list**

- migrate myvm back to server1

## 7. Experiment with liveness using ping command

- in server1, you can recover the console to myvm
  - sudo xl console myvm**
- **To disconnect from the console use this keyboard combination <Ctrl + shift + (>>**
- From server1, we will use ping command to check that the vm is running during the migration
  - ping <myvm IP address>**
- migrate myvm from server1 to server2
  - the ping should not be interrupted

**Q10. Does live migration mean zero downtime? Explain using your measurements.**

## 8. Application-level availability

Ping measures ICMP reachability, but a VM can respond to ping while an application experiences a visible interruption. Test application-level availability during migration.

Inside myvm:

```
python3 -m http.server 8080
```

From server1 or another reachable machine:

```
while true; do
  curl -s -o /dev/null -w "%{time_total}\n" http://<VM-IP>:8080
  sleep 0.1
done
```

Migrate the VM while the HTTP requests are running. Observe response times and failed requests.

**Q11. Compare the observations obtained with ping and HTTP. Do both methods show the same interruption?**

**Q12. Which measurement better represents the impact experienced by an application user? Why?**